

종합 컨텍스트 마이그레이션 문서

1인 플랫폼 운영을 위한 Claude Code 협업 구조

1. 문서 목적

이 문서는 현재 개발 중인 쇼핑몰 생성 및 호스팅 플랫폼을 1인 체제로 운영하기 위해, Claude Code를 중심으로 AI/데이터/운영 워커들과 유기적으로 협업하는 구조를 정의한다.

목표는 다음과 같다.

- 사람을 추가 채용하지 않고도 초기 운영을 가능한 범위까지 자동화한다.
- 개발, 운영, 기획, 데이터 분석을 하나의 일관된 체계로 묶는다.
- 반복 작업, 모니터링, 상태 점검, 일정 관리, 장애 추적을 AI 워커로 분산한다.
- 모든 변화는 사람이 최종 승인하는 구조를 유지한다.
- 초기에는 안정성과 가시성을 우선하고, 자동 실행은 제한적으로 적용한다.

2. 현재 전제

현재 플랫폼은 다음과 같은 전제로 개발되고 있다.

- 1인 개발 체제
- 쇼핑몰 생성 및 호스팅 SaaS
- Claude Code를 중심으로 개발 진행
- Cloudflare / Fly / Neon을 활용한 운영
- 코드 작성, 구조 설계, 운영 판단, 배포 전략을 혼자 수행해야 함
- 향후 입점 쇼핑몰 수가 늘어나면 운영 복잡성이 급격히 증가할 가능성이 높음

따라서 핵심 과제는 단순 개발 속도가 아니라,
“혼자서도 운영 가능한 구조를 먼저 만드는 것”이다.

3. 핵심 운영 원칙

3-1. 사람은 최종 승인자

AI가 제안하고 정리할 수는 있어도, 아래 항목은 사람이 최종 승인한다.

- 프로덕션 배포
- 장애 복구 방식
- 가격/정책 변경
- 이벤트 시작/종료
- DB 구조 변경
- 자동화 범위 확대

3-2. AI는 역할별로 분리

AI 한 개가 모든 것을 처리하지 않는다.
역할을 나눠야 품질과 추적성이 올라간다.

3-3. 운영은 코드보다 제어층이 우선

핵심은 기능을 빨리 더 만드는 것이 아니라,
운영자가 코드 없이 통제할 수 있는 구조를 먼저 만드는 것이다.

3-4. 실시간보다 “지속 가능한 관찰”

24시간 완전 자동 복구보다,
이상 징후를 정확히 감지하고 사람이 빠르게 판단할 수 있는 구조가 우선이다.

3-5. 자동 실행은 단계적으로

초기에는 “분석 → 요약 → 제안 → 승인 요청”까지만.
검증된 작업만 제한적으로 자동 실행한다.

4. 전체 구조 개요

[최종 의사결정자: 나]



| 승인 / 우선순위 / 전략



[AI Chief of Staff]

├ PM AI

├ Dev AI

├ QA AI

├ Ops AI

├ Data AI

├ SRE AI

└ CS AI



[실행 계층]

├ Cloudflare: 엣지, 접근제어, 스케줄, 큐, 알림 트리거

├ Fly: 앱 실행, 워커, 백그라운드 작업

└ Neon: 운영 DB, 테스트 브랜치, 스테이징 브랜치

이 구조의 핵심은 다음과 같다.

- Claude Code는 개발과 구조 설계의 중심이다.
- AI 워커들은 각자 전문 역할만 수행한다.
- Cloudflare/Fly/Neon은 실행과 운영을 분리한다.
- 나는 최종 승인과 전략 판단에 집중한다.

5. 역할 정의

5-1. 나

역할: 최종 의사결정자, 사업 판단자, 승인자

책임:

- 무엇을 만들지 결정
- 어떤 기능을 우선할지 결정
- 어떤 리스크를 감수할지 결정
- AI 제안을 수용 또는 거절
- 장애 시 최종 대응 방향 승인

5-2. Claude Code

역할: 개발 주도 에이전트이자 구조 설계 보조자

책임:

- 아키텍처 설계 보조
- 개발 태스크 분해
- 구현 우선순위 정리
- 코드 변경안 초안 생성
- 기술적 리스크 식별
- 리팩토링 방향 제안
- 운영 구조와 개발 구조의 연결점 정리

Claude Code는 단순 코더가 아니라,

“기능을 구현하는 동시에 운영 가능한 형태로 구조화하는 실무 파트너”여야 한다.

5-3. PM AI

역할: 일정, 범위, 릴리즈 관리

책임:

- 기능 우선순위 정리
- 릴리즈 일정 관리
- 요구사항 충돌 탐지
- 스코프 과대화 방지
- 마감 전 리스크 경고

5-4. Dev AI

역할: 구현 전략, 코드 변경안, 리팩토링 제안

책임:

- 기술 설계 검토

- 변경 범위 분해
- 구현 순서 제안
- 중복 로직 탐지
- 기술 부채 식별

5-5. QA AI

역할: 테스트, 회귀 체크, 오류 재현

책임:

- 핵심 시나리오 테스트 목록 작성
- 배포 전 회귀 테스트 점검
- 오류 재현 경로 정리
- 사용자 영향도 평가

5-6. Ops AI

역할: 행사, 공지, 배너, 운영 일정 관리

책임:

- 이벤트 상태 감시
- 종료 예정 알림
- 설정 누락 확인
- 운영 정책 충돌 탐지
- 반복 운영 업무 감시

5-7. Data AI

역할: 지표 분석, 성과 추적, 이상 탐지

책임:

- 매출, 전환율, 이탈률 분석
- 이벤트 성과 추적
- 사용자 행동 변화 감지
- 비정상 패턴 탐지
- 주간 리포트 요약

5-8. SRE AI

역할: 장애 감시, 성능 감시, 인프라 추적

책임:

- 서버/DB/API 지연 감지
- 오류율 추적
- 배포 직후 이상 감시
- 로그 패턴 분석
- 복구 우선순위 제안

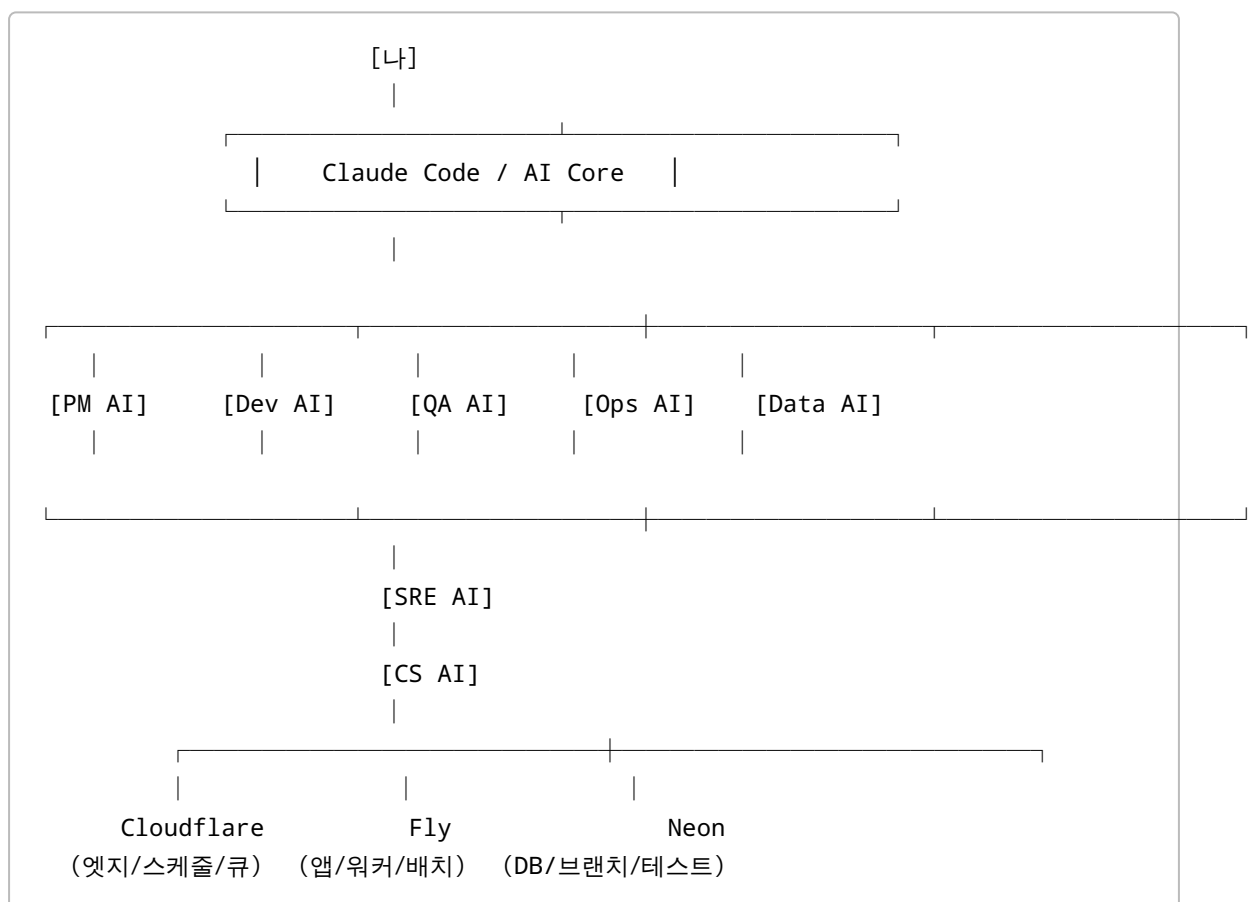
5-9. CS AI

역할: 문의 분류, 반복 이슈 정리

책임:

- 사용자 문의 분류
- 빈도 높은 이슈 추적
- 문제 원인 요약
- 운영팀/개발팀 전달용 요약 생성

6. 권장 운영 조직도



이 조직도는 실제 인력 조직이 아니라,
기능 분리형 작업 체계를 뜻한다.

7. Cloudflare / Fly / Neon의 역할 분담

7-1. Cloudflare

역할: 외부 요청의 관문, 제어와 트리거

주요 활용:

- 엣지에서 요청 수신
- 인증/권한 처리
- 관리자 접근 제어
- 예약 작업 실행
- 큐 기반 비동기 처리
- 이벤트/공지/알림 트리거
- 이상 트래픽 감시

7-2. Fly

역할: 실제 앱 실행과 워커 수행

주요 활용:

- 사용자 앱
- 관리자 앱
- 백그라운드 워커
- 비동기 작업 처리
- 정기 작업 실행
- 알림 서비스
- 무거운 처리 분리

7-3. Neon

역할: 단일 진실원장, DB 브랜칭 기반 격리 환경

주요 활용:

- 운영 데이터 저장
- 스테이징 브랜치
- 프리뷰 브랜치
- 테스트 브랜치
- 배포 전 DB 검증
- 안정적인 롤백 판단

8. Claude Code에게 기대하는 협업 방식

Claude Code는 다음 순서로 움직여야 한다.

8-1. 먼저 구조를 본다

무작정 기능을 만들지 말고,
현재 기능이 운영 구조에 어떤 영향을 주는지 먼저 파악한다.

8-2. 작업을 분해한다

기능 하나를 다음처럼 쪼갬다.

- 사용자 화면
- 관리자 화면
- 데이터 저장
- 운영 정책
- 예외 처리
- 테스트 시나리오
- 로그 및 추적
- 알림 및 복구 흐름

8-3. 위험을 먼저 말한다

Claude Code는 구현 가능한 부분만 말하지 말고, 아래도 함께 알려야 한다.

- 구현 난이도
- 운영 리스크
- 유지보수 비용
- 확장 시 병목
- 테스트 누락 가능성

8-4. 자동화보다 추적성을 우선한다

당장 자동으로 해결하는 것보다, 나중에 원인을 추적할 수 있는 구조를 우선한다.

8-5. 변경은 항상 되돌릴 수 있어야 한다

모든 변경은 롤백 가능성을 고려해야 한다.

9. 운영 루프

혼자 운영하기 위해서는 하루 루프가 필요하다.

아침

- 전날 장애 요약
- 실패한 작업 확인
- 이벤트 진행 상황 확인
- 고객 문의 급증 확인

오후

- 오늘 배포 상태 확인
- 주요 지표 확인
- 이벤트/공지/예약 상태 확인

밤

- 예약 작업 점검
 - 다음날 일정 확인
 - 리스크 사전 탐지
 - 미해결 이슈 정리
-

10. AI 워커 간 협업 원칙

AI끼리 대화는 가능하지만, 다음 조건을 따른다.

- 중요한 변경은 사람 승인 필요
- AI는 분석과 제안을 우선
- AI가 서로 중복 확인하지 않도록 공유 상태를 참조
- 각 AI는 자기 역할 밖의 판단을 과도하게 하지 않음
- 충돌 발생 시 PM AI 또는 SRE AI가 정리
- 최종 판단은 나에게 올라와야 함

즉, AI는 자율적이되 독립적이지는 않다.

11. 자동화 단계

단계 1

- 보고만 한다
- 요약만 한다
- 경고만 한다

단계 2

- 분석하고 제안한다
- 승인 요청을 보낸다
- 반복 작업을 반자동화한다

단계 3

- 검증된 작업만 자동 실행한다
- 나머지는 승인 후 실행한다

단계 4

- 워커 간 협업 자동화
- 단, 프로덕션 변경은 여전히 승인 필요

현재는 단계 1~2가 적절하다.

12. 채용 판단 기준

사람을 뽑아야 하는 시점은 아래 조건이 누적될 때다.

- 문의 대응이 개발 시간을 잠식함
- QA가 자동화만으로 커버되지 않음
- 운영 업무가 반복적으로 밀림
- 장애 대응이 상시 부담이 됨
- 기능 개발이 운영에 밀려 멈춤
- AI가 처리할 수 없는 예외가 매일 발생함

그 전까지는 채용보다 시스템화가 우선이다.

13. 비용에 대한 기본 인식

현재 단계에서는 사람 한 명을 채용하는 것보다,
AI/운영/인프라를 적절히 묶는 편이 훨씬 저렴할 가능성이 높다.

중요한 것은 AI 개수 자체가 아니라 다음이다.

- 공용 메모리
- 이벤트 버스
- 로그 체계
- 승인 구조
- 브랜치 분리
- 워커 분리

이 6개가 있어야 비용이 통제된다.

14. Claude Code가 기억해야 할 우선순위

1. 기능보다 구조
 2. 속도보다 추적성
 3. 자동화보다 승인
 4. 확장보다 안정
 5. 단일 AI보다 역할 분리
 6. 배포보다 롤백 가능성
 7. 코딩보다 운영 가능성
-

15. 최종 요약

이 플랫폼의 목표는 단순히 쇼핑몰 생성 기능을 만드는 것이 아니다.
목표는 **1인이 클라우드 위에서 AI와 함께 운영 가능한 SaaS 조직을 구축하는 것**이다.

이를 위해 필요한 것은 다음과 같다.

- Claude Code를 중심으로 한 개발 파트너십
- 역할 분리된 AI 워커 구조
- Cloudflare/Fly/Neon의 분업
- 승인 중심의 운영 체계
- 자동화보다 추적 가능한 제어층
- 사람 채용 전 단계의 최대한의 시스템화

이 문서는 이후 Claude Code와의 협업에서 기준 문서로 사용한다.

새로운 기능을 만들 때마다 이 문서의 원칙과 조직도에 맞는지 먼저 검토한다.